Отчёт по индивидуальному заданию № 1

Архитектура компьютера и операционные системы

Вариант 9. Магазин с несколькими отделами

ФИО: Жиляков Роман Евгеньевич

Группа: БПИ252

1. Условие задания

В магазине работают три отдела, в каждом находится один продавец. Покупатель выбирает один или несколько различных отделов и порядок их посещения.

Если продавец занят, покупатель становится в очередь соответствующего отдела. После завершения обслуживания покупатель переходит в следующий отдел своего маршрута. Выполнив весь план покупок, он покидает магазин.

Необходимо реализовать последовательную имитацию работы магазина на языке C для Linux. Программа должна отображать действия покупателей и продавцов, сохранять журнал работы и поддерживать обычное завершение и прерывание.

2. Описание модели

Программа выполняется в одном процессе. Потоки и дочерние процессы не используются. Действия участников последовательно обрабатываются в общем цикле моделирования.

В модели предусмотрены три отдела и не более 100 покупателей. Каждый покупатель посещает от одного до трёх различных отделов. Маршрут задаётся во входном файле.

Модельное время измеряется целыми тактами. Первый покупатель появляется в момент 0, каждый следующий – через заданный интервал:

Время появления покупателя $i = (i - 1) \times INTERVAL$

Продолжительность обслуживания выбирается в заданном для отдела диапазоне. Если минимальное и максимальное значения совпадают, длительность обслуживания постоянна.

Очереди обслуживаются по правилу FIFO: первым обслуживается покупатель, который раньше встал в очередь. Для каждого отдела используется массив на 100 номеров покупателей. Этого достаточно, поскольку каждый покупатель посещает один отдел не более одного раза.

Покупатель находится в одном из пяти состояний:

Состояние	**Значение**
NOT_ARRIVED	Ещё не пришёл
WAITING	Ожидает в очереди
SERVING	Обслуживается
LEFT	Выполнил маршрут и вышел
REJECTED	Не допущен после закрытия входа

При наступлении времени CLOSE вход закрывается. Закрытие обрабатывается до появления новых покупателей этого такта. Поэтому покупатель, время появления которого совпадает с CLOSE, не допускается в магазин.

Уже вошедшие покупатели после закрытия продолжают обслуживание и полностью выполняют свои маршруты. Покупатели, которые не успели войти, отмечаются в журнале как не допущенные.

Если все покупатели вышли раньше закрытия, программа ожидает заданного времени закрытия.

3. Алгоритм работы

На каждом такте выполняются следующие действия:

1. Закрытие входа, если наступило время CLOSE.
2. Завершение обслуживаний и переход покупателей в следующие отделы.
3. Появление новых покупателей.
4. Начало обслуживания в свободных отделах.
5. Проверка условия завершения.
6. Необязательная реальная задержка и переход к следующему такту.

При одновременных событиях отделы обрабатываются по номеру. Новые покупатели также обрабатываются по номеру. Переходы между отделами выполняются раньше новых прибытий.

Если продавец свободен, обслуживание начинается на текущем такте после обработки поступлений. Если продавец занят или перед покупателем есть другие запросы, покупатель ожидает в очереди.

Программа поддерживает следующие правила:

- продавец обслуживает только одного покупателя одновременно;
- покупатель находится только в одном месте;
- отделы в маршруте не повторяются;
- следующий отдел посещается после завершения обслуживания в предыдущем;
- обычный выход покупателя происходит после выполнения всего маршрута.

Обычный запуск завершается, поскольку время закрытия конечно, количество покупателей ограничено, а каждый маршрут содержит не более трёх обслуживаний конечной длительности.

4. Состав проекта

Основные файлы находятся в папке hw1.

Файл или каталог	**Назначение**
main.c	Исходный код программы
Makefile	Сборка, запуск тестов и удаление исполняемого файла
tests/*.txt	Входные данные для проверок
tests/run.sh	Скрипт автоматического тестирования

Файл или каталог	Назначение
report.pdf	Итоговый отчёт

При сборке создаётся исполняемый файл shop. Журналы запусков сохраняются в каталоге logs. Исполняемый файл и автоматически полученные журналы не входят в комплект исходных файлов.

5. Интерфейс и формат входных данных

Запуск программы:

./shop INPUT_FILE LOG_FILE DELAY_MS SEED

Аргументы:

Аргумент	Назначение
INPUT_FILE	Путь к входному файлу
LOG_FILE	Путь к журналу
DELAY_MS	Реальная задержка между тактами в миллисекундах
SEED	Начальное значение генератора случайных чисел

Входной файл и журнал должны быть обычными файлами. Каталог для журнала должен существовать.

Реальная задержка не изменяет модельное время и результаты обслуживания. Для автоматических тестов используется задержка 0.

Одинаковый SEED обеспечивает повторяемость случайного сценария при одинаковых входных данных и реализации стандартной библиотеки.

Допустимые значения:

Параметр	Диапазон
Количество покупателей N	От 0 до 100
Интервал появления INTERVAL	От 0 до 100000 тактов
Время закрытия CLOSE	От 0 до 100000 тактов
Время обслуживания MIN, MAX	От 1 до 10000 тактов, MIN ≤ MAX
Длина маршрута K	От 1 до 3
Номера отделов	От 1 до 3, без повторений
Задержка DELAY_MS	От 0 до 1000 мс
SEED	От 0 до INT_MAX
Размер входного файла	Не более 65534 байт

Формат входного файла:

N INTERVAL CLOSE

MIN1 MAX1

MIN2 MAX2

MIN3 MAX3

K D1 ... DK

...

После параметров отделов указываются N маршрутов покупателей.

Пример:

3 0 5

2 2

2 2

2 2

2 1 2

1 1

2 3 1

В этом примере:

- все три покупателя появляются в момент 0;
- вход закрывается в момент 5;
- обслуживание в каждом отделе занимает два такта;
- покупатель 1 посещает отделы 1 и 2;
- покупатель 2 посещает отдел 1;
- покупатель 3 посещает отделы 3 и 1.

Программа отклоняет неверные числа, значения вне диапазона, повторные отделы, неполные и лишние данные, переполнение и нулевые байты во входном файле.

6. Сборка и запуск

Из корня проекта:

make -C hw1

make -C hw1 test

Ручной запуск:

cd hw1

mkdir -p logs

./shop tests/normal.txt logs/manual.log 200 1

Вывод справки:

./shop --help

Удаление исполняемого файла:

make clean

Для сборки необходим GCC. Для тестирования используются Bash, awk и стандартные утилиты GNU/Linux, включая timeout.

Программа собирается в стандарте C11 с параметрами:

-D_POSIX_C_SOURCE=200809L -Wall -Wextra -Wpedantic -O2

7. Ввод-вывод и журнал

Входной файл читается через файловый дескриптор с помощью open, read и close. Чтение учитывает частичный результат, конец файла и прерывание системного вызова.

Числа разбираются с помощью strtol, после чего проверяются диапазон и границы токена.

События записываются в стандартный вывод и журнал через write. Обрабатываются частичная запись и ошибка EINTR. Диагностические сообщения направляются в стандартный поток ошибок.

Перед очисткой журнала программа проверяет, что он является обычным файлом и не совпадает с входным файлом. Проверка учитывает символические и жёсткие ссылки. Это защищает входные данные от перезаписи.

В журнале отражаются:

- вход покупателя и его маршрут;
- переход между отделами;
- постановка в очередь;
- начало и завершение обслуживания;
- выход покупателя;
- закрытие входа и отказ во входе;
- причина завершения;
- итоговая статистика и незавершённые маршруты.

Каждое событие содержит модельное время в формате [t=...].

8. Завершение программы

При обычном завершении вход закрыт, все вошедшие покупатели вышли, очереди пусты, продавцы свободны.

Для прерывания предусмотрены сигналы SIGINT и SIGTERM. При ручном запуске SIGINT можно отправить сочетанием клавиш Ctrl+C.

Обработчик сигнала только сохраняет его номер во флаг типа volatile sig_atomic_t. Основной код обнаруживает флаг, завершает текущий такт, сохраняет итоговую статистику и закрывает файлы.

После получения сигнала дальнейший вывод в терминал пропускается, а итог сохраняется в журнале. Для стандартного вывода в канал используется неблокирующий режим и ожидание через poll. Поэтому заполненный канал не препятствует обработке прерывания.

При прерывании покупатели, оставшиеся внутри, не считаются успешно завершившими маршрут. В журнале сохраняются их фактические состояния.

Код завершения	**Значение**
0	Обычное завершение или успешный вывод справки
1	Ошибка аргументов, данных, файлов или вывода
130	Прерывание сигналом SIGINT
143	Прерывание сигналом SIGTERM

Проверяемые балансы покупателей:

Вошли = вышли + внутри

Всего = вошли + не допущены + ещё не пришли

После обычного завершения число покупателей внутри и число ещё не пришедших равны нулю.

9. Тестирование и результаты

Проверка выполнена в Linux с GCC 13.3.0. Строгая сборка с дополнительным параметром -Werror завершилась без предупреждений.

В проверенной версии скрипта успешно выполнены 42 группы тестов. Некоторые группы включают несколько запусков: например, проверку 20 значений SEED.

Скрипт анализирует журнал и проверяет порядок FIFO, маршруты, занятость продавцов, время окончания обслуживания, однократность входа и выхода, а также итоговую статистику.

Сценарий	**Полученный результат**
normal	Завершение в момент 6; вошли и вышли 3 покупателя; покупки по отделам: 3, 1, 1
closing	Завершение в момент 8; вошли и вышли 2 покупателя; один не допущен
empty	Завершение в момент 5; покупателей нет
one	Единственный покупатель завершил маршрут и вышел
route	Маршрут 3 → 1 → 2 завершён в момент 6
fifo	Четыре покупателя обслужены в порядке 1, 2, 3, 4; завершение в момент 8
parallel	В трёх отделах обслуживание началось в момент 0; завершение в момент 3
boundary	Покупатель, появившийся в момент закрытия, не допущен
closed	При закрытии в момент 0 никто не вошёл; зарегистрированы два отказа
100 покупателей	Все вышли; выполнено 300 покупок; завершение в момент 102

Сценарий	**Полученный результат**
random, 20 значений SEED	Все восемь покупателей вышли; покупки по отделам: 6, 5, 6
Разные реальные задержки	Модельные события и их время совпали
SIGINT, SIGTERM	Получены коды 130 и 143; состояния оставшихся покупателей сохранены

Проверены ошибочные данные: повтор отдела, неверный номер отдела, перевёрнутый диапазон обслуживания, нулевая длительность, отрицательные значения, переполнение, повреждённый числовой токен, обрыв и лишние данные.

Также проверены отсутствующий файл, каталог вместо файла, слишком большой файл, нулевой байт, неверные аргументы, невозможность открыть журнал и ошибка записи в /dev/full.

Отдельные проверки подтвердили защиту входного файла от перезаписи через обычный путь, символическую и жёсткую ссылки.

Для SIGINT и SIGTERM проверено завершение при заполненном канале стандартного вывода. Программа завершалась без зависания и сохраняла итог в журнале.

Все 42 группы также прошли с UndefinedBehaviorSanitizer. Четыре обычных сценария и два сценария прерывания при заполненном канале дополнительно проверены с AddressSanitizer совместно с UndefinedBehaviorSanitizer. Ошибок памяти и неопределённого поведения в этих проверках не обнаружено.

10. Примеры работы

Основной сценарий:

[t=0] Начало: покупатель=1 отдел=1 окончание=2

[t=0] Начало: покупатель=3 отдел=3 окончание=2

[t=2] Начало: покупатель=2 отдел=1 окончание=4

[t=2] Начало: покупатель=1 отдел=2 окончание=4

[t=4] Выход: покупатель=2 план=выполнен

[t=4] Выход: покупатель=1 план=выполнен

[t=4] Начало: покупатель=3 отдел=1 окончание=6

[t=5] Вход закрыт

[t=6] Выход: покупатель=3 план=выполнен

[t=6] Итог: причина=завершено

[t=6] Отдел: номер=1 покупок=3 очередь=0 обслуживается=0

[t=6] Отдел: номер=2 покупок=1 очередь=0 обслуживается=0

[t=6] Отдел: номер=3 покупок=1 очередь=0 обслуживается=0

Обслуживание после закрытия:

[t=3] Вход закрыт

[t=3] Отказ: покупатель=3 причина=вход_закрыт плановое_прибытие=4

[t=4] Выход: покупатель=1 план=выполнен

[t=8] Выход: покупатель=2 план=выполнен

[t=8] Итог: причина=завершено

Прерывание во время обслуживания:

[t=0] Итог: причина=SIGINT

[t=0] Отдел: номер=1 покупок=0 очередь=2 обслуживается=1

[t=0] Отдел: номер=2 покупок=0 очередь=0 обслуживается=0

[t=0] Отдел: номер=3 покупок=0 очередь=0 обслуживается=0

[t=0] Остался: покупатель=1 состояние=обслуживание отдел=1 выполнено=0 из=3

[t=0] Остался: покупатель=2 состояние=очередь отдел=1 выполнено=0 из=3

[t=0] Остался: покупатель=3 состояние=очередь отдел=1 выполнено=0 из=3

В последнем примере первый покупатель обслуживается, два других ожидают. Их маршруты не отмечаются как выполненные.

11. Ограничения модели

Количество отделов фиксировано и равно трём. Количество покупателей ограничено 100. Маршруты и интервал появления задаются входным файлом.

Модель конечная, отдельного бесконечного режима нет. Прерывание доступно для любого запущенного моделирования.

Ассортимент, стоимость товаров и оплата не моделируются. Случайность используется только для выбора длительности обслуживания. Большая реальная задержка увеличивает время демонстрации, поэтому в автоматических тестах она отключается.

12. Использование ИИ при тестировании и оформлении работы

ChatGPT использовался для подготовки тестовых данных и скрипта проверки, анализа результатов запусков и поиска граничных случаев. Проверялись порядок обслуживания FIFO, выполнение маршрутов, закрытие входа, некорректные данные и завершение по сигналам SIGINT и SIGTERM.

ИИ также помог составить отчёт: описать модель, интерфейс программы, способы сборки, запуска и завершения, а также результаты тестирования. Дополнительно были получены пояснения по настройке Git, авторизации на GitHub и отправке проекта через PowerShell.

Запросы:

Тестирование:

Подготовь простые тестовые данные и скрипт проверки программы. Проверь FIFO, порядок посещения отделов, обслуживание после закрытия, пустой магазин, максимальное число покупателей и неверные входные данные. Укажи ожидаемые результаты.

Анализ результатов:

Проанализируй приложенный вывод тестов. Объясни причины ошибок. Отдельно проверь корректное завершение по SIGINT и SIGTERM и сохранение итогового журнала.

Подготовка отчёта:

Составь отчёт по условию задания, исходному коду и фактическим результатам тестирования. Включи описание модели и интерфейса, команды сборки и запуска, способы завершения и результаты работы. Оставь поля только для ФИО и группы. Не добавляй неподтверждённые сведения.

Работа с GitHub через PowerShell:

Дай полные команды PowerShell для настройки репозитория и отправки проекта в репозиторий acos на GitHub. Работа должна находиться в папке hw1 и ветке hw1. Нужны один итоговый коммит и один PR из hw1 в master. Объясни порядок авторизации и проверки файлов перед отправкой.